

Product Requirements Document (PRD)

APIx — Real-Time Airfare Price Index for India

Team: CodeCrew | **Event:** Smart India Hackathon 2026 | **PS ID:** SIH26056 **Document version:** 1.0 | **Status:** Prototype / Hackathon build, validated **Official title:** *Development of a Real-time Airfare Price Index for India through Automated Web Scraping of Airline and OTA Portals for Augmentation of the Consumer Price Index (CPI)*

1. Executive Summary

APIx is an automated data pipeline that scrapes live airfare data from major Indian airlines and OTAs, cleans and decomposes it into its economic components, and computes a DGCA-traffic-weighted, econometrically rigorous price index for domestic air travel. The index is back-tested against DGCA's own published fare data ($r = 0.9997$, $\text{MAPE} = 2.41\%$) and exposed via a REST API and dashboard for consumption by NSO, RBI, and MoSPI's eSankhyiki platform — with a secondary regulatory-compliance capability (the DGCA Watchdog) for DGCA's Tariff Monitoring Unit. The goal is to give India's Consumer Price Index a faster, more representative airfare signal than the current manual, monthly survey process can provide.

2. Problem Statement

India's CPI Transport & Communication sub-group prices airfares through manual, low-frequency, counter-collected quotes gathered by MoSPI's eSankhyiki system. This has three structural weaknesses:

Weakness	Impact
Lag	Official figures trail real market conditions by roughly 45 days.
Coverage	Manual counter quotes miss the dynamic, revenue-managed online pricing (which can swing 200–400% with demand) that now accounts for the large majority of ticket sales.
	A monthly snapshot cannot capture short-term shocks — a fuel-price spike, a

Frequency	festival demand surge, or a sudden fare war.
------------------	--

Airfare sits inside a highly weighted, policy-sensitive basket (Transport = 8.59% of urban CPI) despite being a small absolute share of headline CPI (~0.41%), meaning a mispriced or lagging reading distorts inflation measurement disproportionately to the cost of fixing it.

3. Goals & Objectives

Primary goal: Produce a daily, auditable, DGCA-weighted airfare price index accurate enough to meaningfully augment (not replace) the official CPI airfare sub-component.

Objectives:

1. Achieve a validated correlation of $r \geq 0.88$ and **MAPE** < **5.0%** against DGCA's official benchmark fare data (achieved: $r = 0.9997$, MAPE = 2.41%, RMSE = ₹197.47 — all comfortably inside the ≥ 0.88 / <5.0% / <₹450 targets).
2. Cover a DGCA-traffic-representative basket of domestic routes rather than an arbitrary sample (achieved: 30 corridors, ~69% of national passenger traffic, 18 airports).
3. Deliver the index with a reporting lead of ~45 days versus the official cycle.
4. Provide a machine-consumable export compatible with MoSPI's eSankhyiki / Price Statistics Division ingestion format.
5. Demonstrate the pipeline is ethically and legally sourced (robots.txt compliance, rate limiting) so it is deployable in good faith.
6. (Stretch, delivered) Extend the same pipeline into a regulatory tool — the DGCA Watchdog — that flags predatory/statutory-cap-breaching fares in near real time.

4. Target Users & Stakeholders

User	Need	How APIx serves it
NSO (National Statistical Office)	Faster, more representative airfare input into CPI compilation	Daily/weekly/monthly index, eSankhyiki-compatible export
RBI	Timely inflation signal for monetary policy	Augmented Transport CPI with quantified basis-point gap vs.

		official series
MoSPI / eSankhyiki	A schema-compatible data feed, not raw scraped data	/esankhyiki/export endpoint formatted to PSD/COICOP conventions
DGCA Tariff Monitoring Unit	Near-real-time detection of predatory/surge pricing	The DGCA Watchdog's 4-tier alert system across 30 corridors
Aviation analysts / researchers (secondary)	Route-level fare and demand insight	Public dashboard, open API, decomposition and lead-time-curve endpoints

5. Scope

5.1 In scope (delivered)

- An ethical scraping framework (robots.txt compliance, per-domain rate limiting, UA rotation) covering 5 airlines and 5 OTAs. **Current-build note:** the airline path genuinely attempts one live HTTP request before falling back; the OTA path and the Playwright-based scraper are synthetic-only in the present build (see the Tech Stack document, Section 3, for the exact breakdown). The calibrated synthetic pricing engine is therefore the dominant data source today, not an occasional fallback.
- Cleaning pipeline: deduplication (direct-airline priority), stratified IQR outlier removal, non-zero sold-out imputation, fare decomposition (base fare / GST / UDF / PSF / OTA fee).
- A Two-Tier Jevons-Laspeyres Hybrid index formula, computed daily/weekly/monthly, over a 30-corridor, DGCA-weighted basket sampled across 5 advance-purchase windows (T+1 to T+45).
- A 90-day backtest against DGCA's own benchmark fare data.
- An eSankhyiki/MoSPI augmentation module (COICOP 2018-aligned, 2012=100 rebasing) with a National Data Warehouse export feed.
- A DGCA Price-Ceiling & Festive-Surge Watchdog (4-tier classification, per-corridor advisory caps, 4 tracked festive windows).
- A 12-endpoint FastAPI service and a 9-tab Streamlit dashboard.
- A 27-test automated pytest suite across 7 modules.

- Containerized deployment (Docker/docker-compose), reverse-proxied via Nginx, demoed on a live cloud instance.

5.2 Out of scope (explicitly not attempted)

- Replacing NSO/MoSPI's CPI methodology or compilation authority.
- Capturing actually-transacted fares (only posted/quoted fares are scraped; corporate-desk or call-center bookings are not visible).
- Coverage of international routes, cargo, or non-scheduled/charter aviation.
- Independently validated statutory-cap or surge thresholds for the Watchdog (currently configured defaults, pending DGCA sign-off).
- High-availability, multi-region production infrastructure (current deployment is a single-instance demo).

6. Functional Requirements

ID	Requirement	Priority	Status
FR-1	System shall attempt live fare retrieval from ≥ 5 airlines and support ≥ 5 OTA sources	Must	Partial — airline scraper attempts one real HTTP request with fallback; OTA scraper and the Playwright class are synthetic-only in the current build
FR-2	Every scrape request shall be gated by a robots.txt check and per-domain rate limit	Must	Done
FR-3	System shall fall back to a calibrated synthetic pricing engine when a source is blocked	Should	Done
FR-4	System shall deduplicate quotes, prioritizing direct-airline over OTA sources	Must	Done
FR-5	System shall remove statistical outliers per route-per-window (stratified IQR), not globally	Must	Done
FR-	System shall impute (not zero-fill or silently		

6	drop) sold-out flight data	Must	Done
FR-7	System shall decompose every fare into base fare, GST, UDF, PSF, and OTA convenience fee	Must	Done
FR-8	System shall compute a DGCA-weighted composite index using real passenger-traffic weights	Must	Done
FR-9	System shall sample each route across 5 advance-purchase windows (T+1, T+7, T+15, T+30, T+45)	Must	Done
FR-9b	System shall sample each route 3 times daily at fixed intraday epochs (08:00 / 14:00 / 20:00 IST), weighted 0.20 / 0.30 / 0.50 respectively, to eliminate time-of-day pricing bias	Should	Done
FR-10	System shall back-test the computed index against official DGCA benchmark data	Must	Done
FR-11	System shall expose index values, history, decomposition, and metadata via a REST API	Must	Done
FR-12	System shall provide an interactive dashboard for exploring the index, corridors, and backtest results	Must	Done
FR-13	System shall export an eSankhyiki/MoSPI-compatible augmented-CPI feed	Should	Done
FR-14	System shall classify fares into regulatory tiers (Compliant/Elevated/Predatory/Cap Breach) per corridor	Could	Done
FR-15	System shall support on-demand, manually-triggered scrape runs via API	Could	Done
FR-16	System shall be independently deployable via containerization	Should	Done

Note on FR-8 / index formulas: every daily computation actually returns **five** index

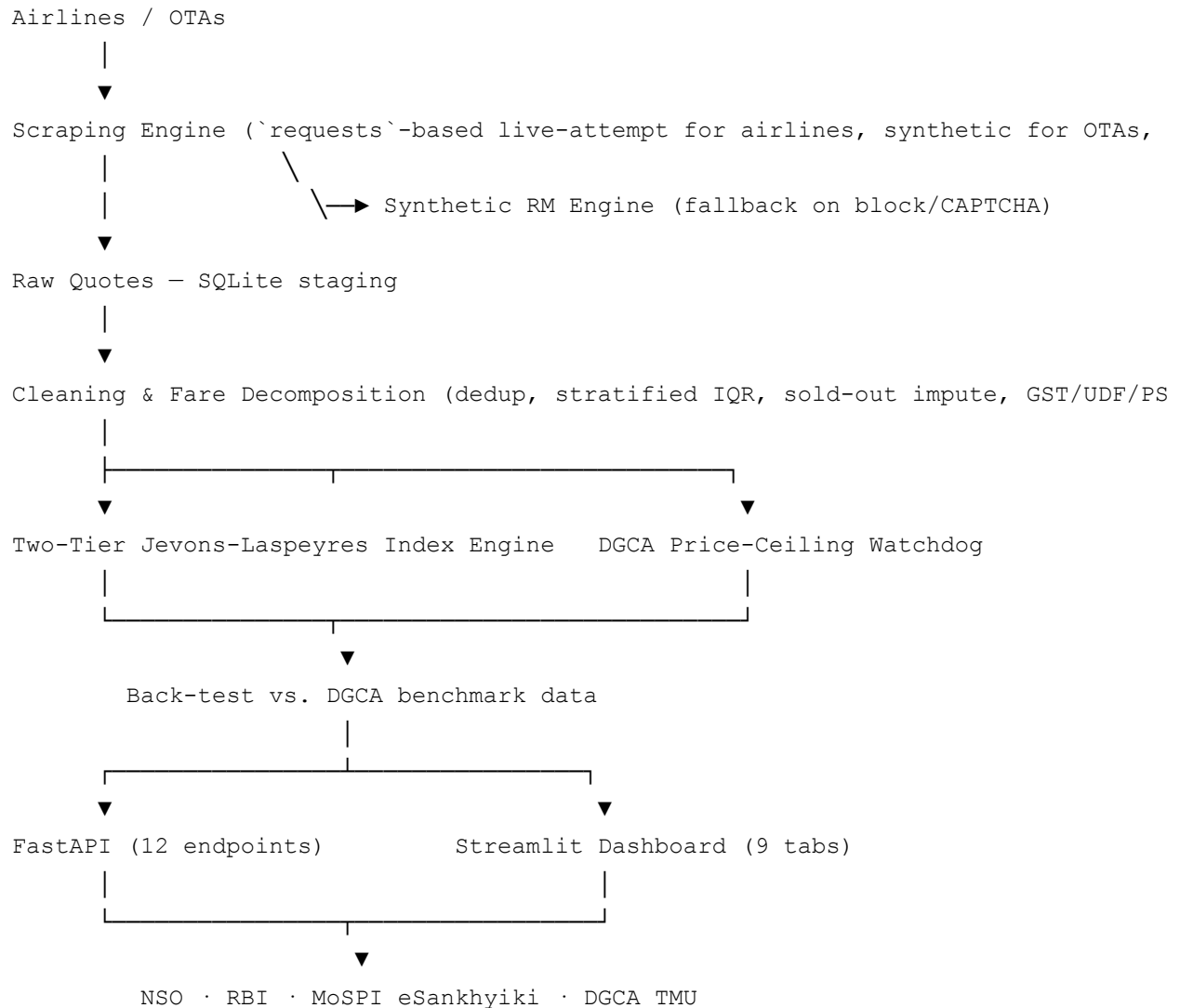
fields — `two_tier_index` (the official headline "champion" formula), `laspeyres_index`, `paasche_index`, `fisher_index`, and `jevons_index` — all persisted to the database and exposed via the API (verified directly in `engine.py` and asserted by `test_api.py`'s `test_latest_index_endpoint`). However, by design, `laspeyres_index` and `paasche_index` are both computed by calling the *same* underlying two-tier aggregation function with identical inputs (`paasche_index`'s own code comment explains that true Paasche needs real-time flyer counts airlines don't disclose), and `fisher_index` is `sqrt(laspeyres * paasche)` — so in practice **`laspeyres_index`, `paasche_index`, and `fisher_index` will always compute to the same value as each other**. Only `jevons_index` (a genuinely separate weighted-log-average calculation) and `two_tier_index` (which uses Jevons-smoothed elementary prices rather than simple medians) are mathematically distinct. This is intentional, documented behavior, not a bug — the four classical formulas are retained for transparency/comparability, but the two-tier hybrid is the one formula the project treats as authoritative.

7. Non-Functional Requirements

Category	Requirement
Accuracy	Index must achieve $r \geq 0.88$, $\text{MAPE} < 5.0\%$, $\text{RMSE} < ₹450$ against DGCA benchmark data.
Ethics/Compliance	No scrape request may proceed if disallowed by the target's robots.txt; all requests must be rate-limited with jitter.
Resilience	A single source's scraping failure must not block or corrupt the index for other sources/routes.
Modularity	Scraping, cleaning, indexing, validation, and serving layers must be independently testable and replaceable.
Auditability	Every index value must be traceable to its underlying weights, formula, and decomposition logic — no black-box outputs.
Performance	API responses for index/metadata endpoints should be near-instant (SQLite-backed, no live scraping on the request path).
Portability	The full stack must run reproducibly via Docker on a fresh machine with no manual environment setup beyond <code>docker-compose up</code> .
Extensibility	Adding a new route/corridor must require only a config/weights change, not a code change to the scraper, cleaner, or index engine.

Documentation	All API endpoints must self-document via OpenAPI/Swagger.
----------------------	---

8. High-Level Architecture



Tech stack: Python · FastAPI · Streamlit + Plotly · SQLite · `requests` + calibrated synthetic pricing engine (Playwright/Scrapy scaffolded for a future live-scraping upgrade) · pandas / NumPy / SciPy · pytest · Docker + Nginx.

9. Data Requirements

Data		
-------------	--	--

element	Source	Detail
Route basket & weights	DGCA domestic passenger-traffic data (cross-verified via the open India Aviation Traffic dataset)	30 corridors, ~69% of national traffic; top weight BOM–DEL at 11.51%
Airport fee schedule	Configured per airport	UDF ₹300–480, PSF ₹91–110, across 18 airports
GST rates	CBIC published slabs	5% economy, 12% business/premium
OTA convenience fees	Per-platform config	₹0 (EaseMyTrip) to ₹399 (Yatra)
Advance-purchase window weights	Modeled booking-lead-time distribution	T+1 10%, T+7 25%, T+15 30%, T+30 25%, T+45 10%
Intraday sampling epochs	3-wave daily schedule (<code>scheduler.py</code>)	08:00 IST (weight 0.20, opening inventory), 14:00 IST (weight 0.30, corporate booking cycle), 20:00 IST (weight 0.50, ~50% of daily retail bookings)
DGCA benchmark (validation)	DGCA official average-fare data	3 monthly snapshots × 6 named routes = 18 comparison points
CPI weighting chain	MoSPI PSD / COICOP 2018	Transport = 8.59% of urban CPI; airfare ≈ 0.41% of headline CPI
Watchdog thresholds	Configured (pending DGCA validation)	Elevated ≥2.0x, Predatory ≥2.5x, per-corridor caps ₹8,000–₹18,000

10. Success Metrics / KPIs

Metric	Target	Achieved
Correlation vs. DGCA benchmark (Pearson r)	≥ 0.88	0.9997

MAPE vs. DGCA benchmark	< 5.0%	2.41%
RMSE vs. DGCA benchmark	< ₹450.00	₹197.47
Reporting lead vs. official CPI cycle	Faster than monthly	+45 days
Basket traffic representativeness	As broad as feasible	~69% of national traffic (30 corridors)
Automated test pass rate	100%	27/27 passing
Ethical scraping compliance	100% of requests robots.txt-gated	Enforced in <code>base_scraper.py</code>

11. Assumptions & Constraints

- Posted/quoted online fares are treated as a reasonable proxy for the price a typical consumer faces; actually-transacted fares are not directly observable.
- DGCA's published traffic and fare data are assumed to be an authoritative, if imperfect, ground truth for weighting and validation.
- The project operates at hackathon/demo scale (SQLite, single-instance deployment); production-scale ingestion volumes are out of current scope.
- The synthetic pricing engine is currently the dominant data source in practice, not an occasional fallback — only the airline scraper attempts a live HTTP call at all (to a plausible-but-unconfirmed endpoint path), and it will typically fail over to synthetic data against real production sites. This is architecturally by design (compliance over completeness) but should be stated plainly, not implied to be a minor edge case.
- Watchdog thresholds are engineering defaults pending regulatory sign-off, not DGCA-endorsed values.

12. Risks & Mitigations

Risk	Likelihood	Mitigation
Target sites block	High	Ethical guardrails minimize triggering blocks;

scraping or change layout	(inherent to any scraper)	synthetic engine maintains pipeline continuity; modular design isolates the failure to one source
Basket/weights become stale as routes/traffic shift	Medium	Weights are config-driven, not hardcoded — periodic recalibration is a config update, not a rebuild
Overstating index accuracy to policy stakeholders	Medium	Explicit "trend correlation, not exact-value match" framing; all validation numbers are documented alongside their acceptance thresholds
eSankhyiki export schema drifts from MoSPI's real ingestion format	Medium	Export logic isolated in one module (<code>esankhyiki.py</code>); would need direct MoSPI collaboration to finalize schema
Watchdog thresholds mis-calibrated for real enforcement use	Medium	Thresholds are tunable config values, explicitly flagged as requiring DGCA validation before operational use
Single-instance deployment has no failover	Low (demo-stage acceptable)	Containerized design allows straightforward horizontal scaling if required later

13. Roadmap / Future Scope

1. **Basket expansion** beyond 30 corridors toward fuller national traffic coverage.
2. **Database migration** to PostgreSQL for production-scale ingestion and concurrency.
3. **Official data partnerships** with airlines/OTAs/DGCA to reduce dependence on scraping.
4. **Automated weight recalibration** on a periodic (e.g. quarterly) cycle rather than a fixed snapshot.
5. **Joint validation exercise with MoSPI/NSO** to test the eSankhyiki augmentation methodology against their internal accuracy standards.
6. **DGCA collaboration** to calibrate and formally adopt Watchdog surge/ceiling thresholds.
7. **Generalization** of the scrape → clean → weight → validate → augment pattern to other dynamically-priced CPI sub-components.

14. Glossary

- **CPI** — Consumer Price Index. **COICOP** — Classification of Individual Consumption by Purpose. **UDF/PSF** — User Development Fee / Passenger Service Fee (airport charges). **MAPE/RMSE** — Mean Absolute Percentage Error / Root Mean Square Error. **AP window** — Advance-Purchase window (days before departure). **TMU** — DGCA's Tariff Monitoring Unit. **PSD** — MoSPI's Price Statistics Division.